Programme de Formation — Angular Avancé

Perfectionnement et Nouvelles Fonctionnalités · De Angular 14 vers Angular 21+

Durée	4 jours (28 heures)
Public	Développeurs Angular ayant une première expérience (Angular v14+)
Prérequis	Connaissance de base d'Angular, TypeScript, HTML/CSS
Format	Intra-entreprise, présentiel
Pédagogie	60% pratique / 40% théorique — Travaux pratiques sur chaque module

Formateur Frédéric GAURAT

Objectifs de la formation

- Maîtriser les fonctionnalités avancées d'Angular 21+ (Signals, Standalone, zoneless).
- Savoir migrer un projet Angular 14 vers les versions récentes.
- Implémenter une gestion d'état robuste avec NGRX (store, effects, entity).
- Mettre en place des tests unitaires et E2E professionnels (Vitest, Playwright).
- Sécuriser les routes avec des guards, l'authentification et la gestion des rôles.
- Concevoir des formulaires réactifs avancés, dynamiques et basés sur les signaux.
- Maîtriser RxJS dans un contexte de coexistence avec les Signals.

Programme détaillé

Module 1 — Rappel TypeScript et Architecture Angular *(0,5 jour)*

- Rappels TypeScript essentiels : types avancés, generics, decorators, utility types.
- Architecture applicative Angular : bonnes pratiques de structuration.
- Component-level Lazy Loading et optimisation du bundle.
- Data Binding avancé : ChangeDetectorRef, mode OnPush.
- Introduction au CLI Angular et outillage (Angular DevTools).

Travaux pratiques : Analyser et optimiser une application Angular existante. Identifier les points de performance.

Module 2 — Angular Standalone et Migration v14 vers v21+ *(0,5 jour)*

- Pourquoi les Standalone Components ? Abandon progressif des NgModules.
- Mise en place de composants, directives et pipes standalone.
- Architecture 100% standalone : providers, routing, bootstrap.
- Passage d'une architecture modulaire (v14) vers standalone.
- Processus de migration Angular v14 vers v21/v22 : ng update, schematics, breaking changes.
- Points de vigilance et stratégie de migration progressive.

Travaux pratiques : Migrer un mini-projet de NgModules vers une architecture 100% standalone.

Module 3 — Les Signaux Angular (Signals) *(0,5 jour)*

- Présentation de l'API Signals : `signal()`, définition, lecture et écriture.
- Signaux en lecture seule (`computed()`).
- Les effets (`effect()`) : réagir aux changements.
- Lien avec le monde des observables : `toSignal()`, `toObservable()`.
- Quand utiliser les signaux vs RxJS : guide de décision.

Travaux pratiques : Réécrire un composant utilisant des observables en remplaçant ces derniers par des signaux et dérivés.

Module 4 — RxJS et Programmation Réactive *(0,5 jour)*

- Fondamentaux RxJS : Observable, Subject, BehaviorSubject.
- Opérateurs essentiels : map, filter, switchMap, mergeMap, combineLatest, debounceTime.
- Gestion des souscriptions et stratégies d'unsubscribe (`takeUntilDestroyed`).
- Patterns avancés : retry, error handling, caching.
- Coexistence RxJS et Signals dans un projet Angular moderne.
- Maintenir du code RxJS existant tout en adoptant les signaux.

Travaux pratiques : Créer un service de données réactif combinant RxJS et Signals.

Module 5 — Fonctionnement Interne et Optimisation *(0,25 jour)*

- ZoneJS : concepts et mécanisme de détection de changement.
- Optimisation des cycles de rendu : OnPush, exécution hors ZoneJS.
- Angular zoneless (expérimental) : `provideExperimentalZonelessChangeDetection()`.
- Lazy loading avancé : routes, composants, defer blocks.
- Stratégies de performance : trackBy, memoization, Virtual Scrolling.

Travaux pratiques : Profiler et optimiser une application Angular avec Angular DevTools.

Module 6 — Routing Avancé, Authentification et Gestion des Rôles *(0,5 jour)*

- Configuration de routes avancée : nested routes, named outlets.
- Route Guards fonctionnels (`canActivate`, `canActivateChild`, `canDeactivate`, `canMatch`).
- Pré-chargement de données avec Resolve.
- Protection des routes par authentification (JWT, session).
- Gestion des rôles et permissions : guards conditionnels.
- Intégration avec un back-end Spring Boot (contexte projet).
- Lazy loading de routes et preloading strategies.

Travaux pratiques : Implémenter un système d'authentification avec guards, rôles et resolve sur une application multi-vues.

Module 7 — Formulaires Réactifs Avancés et Signal Forms *(0,5 jour)*

- ReactiveFormsModule : FormControl, FormGroup, FormArray, FormBuilder.
- Typed Forms (Angular 14+) : formulaires typés statiquement.
- Validation synchrone et asynchrone personnalisée.
- Signal Forms (expérimental, `@angular/forms/signals`) : la nouvelle API de formulaires basée sur les signaux.
- Formulaires dynamiques : génération à partir de métadonnées.
- Abstraction de composants métier de formulaire réutilisables.

Travaux pratiques : Concevoir un formulaire complexe dynamique en Signal Forms avec validations croisées et composants réutilisables.

Module 8 — Gestion d'État avec NGRX *(0,5 jour)*

- Architecture Flux/Redux appliquée à Angular.
- NGRX Store : actions, reducers, selectors.
- NGRX Effects : gestion des effets de bord et appels API.
- NGRX Entity : gestion simplifiée des collections.
- NGRX et Signals : intégration avec `selectSignal`.
- Bonnes pratiques : structuration du store, feature states.
- DevTools Redux pour le débogage.

Travaux pratiques : Mettre en place un store NGRX complet (CRUD) avec effects pour les appels API REST Spring Boot.

Module 9 — Tests Unitaires et End-to-End *(0,75 jour)*

- Stratégie de test pour une application Angular en contexte projet.
- Tests unitaires avec Vitest (ou Jest) : configuration et bonnes pratiques.
- Testing Library pour Angular : approche centrée utilisateur.
- Cas de test : composants, services, pipes, guards, resolvers.
- Mocking des dépendances et des appels HTTP (HttpClientTestingModule).
- Tests End-to-End avec Playwright : installation, configuration, bonnes pratiques.
- Gestion des problèmes de proxy en environnement entreprise.
- Code Coverage et intégration CI/CD.
- Approches TDD et BDD appliquées à Angular.

Travaux pratiques : Développer une fonctionnalité complète en TDD. Écrire des tests E2E avec Playwright sur un parcours utilisateur.

Module 10 — Injection de Dépendances et Composants UI *(0,25 jour)*

- Injection de dépendances avancée : `inject()`, `providedIn`, hiérarchie d'injecteurs.
- Bibliothèques de composants graphiques : Angular Material
- Internationalisation (i18n) : externalisation des chaînes, bonnes pratiques.
- Gestion du pluriel et des formats localisés.

Travaux pratiques : Intégrer Angular Material dans un formulaire et configurer l'internationalisation.